

+ TEMA 12

ANÁLISIS INTEGRADO EN R Y PYTHON

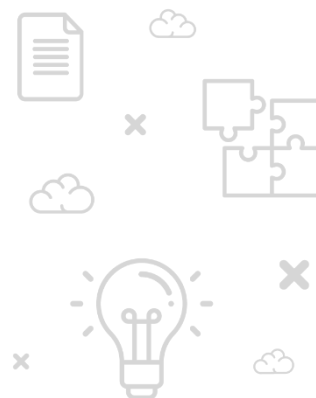
- Comparación de herramientas: ventajas de R vs. Python
- Integración: combinar flujos de trabajo
- Análisis estadístico: pruebas en ambas herramientas
- Minería de datos: algoritmos en R y Python
- Optimización de flujos de trabajo: eficiencia en pipelines de datos





+ INTRODUCCIÓN **DE LA SESIÓN**

- + El análisis de datos moderno exige el uso de herramientas flexibles y potentes que permitan abordar problemas estadísticos y de minería de datos de manera eficiente. En este contexto, R y Python se han consolidado como los lenguajes más utilizados en análisis estadístico y ciencia de datos.
 - + R destaca por su fortaleza en estadística, análisis exploratorio y visualización avanzada, mientras que Python sobresale por su versatilidad, escalabilidad e integración con sistemas productivos. Ambos lenguajes no compiten, sino que se complementan estratégicamente en entornos profesionales.
- La integración de R y Python permite aprovechar lo mejor de cada
- + herramienta dentro de un mismo flujo de trabajo para optimizar tiempos, recursos y calidad de los análisis. Esto resulta clave en proyectos reales de negocio y analítica avanzada.



+

COMPARACIÓN DE
**HERRAMIENTAS: VENTAJAS DE
R VS. PYTHON**

R Y PYTHON

Son lenguajes de programación ampliamente utilizados en el análisis estadístico. Ambos permiten transformar datos en información útil para la toma de decisiones, pero difieren en su origen, enfoque y forma de desarrollo.

R

- Fue creado para el análisis estadístico y la investigación académica.
- Su fortaleza está en la estadística, la econometría y la visualización especializada.

PYTHON

- Es un lenguaje de propósito general que ha incorporado librerías para análisis de datos, machine learning y automatización, lo que lo hace ideal para el uso empresarial.

DIFERENCIAS CLAVE ENTRE R Y PYTHON

ASPECTO	R	PYTHON
Enfoque original.	Estadística.	Lenguaje general.
Curva de aprendizaje.	Media.	Baja a media.
Uso empresarial.	Medio.	Alto.
Automatización.	Limitada.	Muy alta.
Integración con sistemas.	Media.	Excelente.
Visualización.	Muy avanzada.	Clara y flexible.

VENTAJAS DEL DESARROLLO EN PYTHON FRENTE A R

ASPECTO	R	PYTHON
Claridad del flujo.	Alta.	Muy alta.
Integración futura.	Media.	Excelente.
Automatización.	Limitada.	Total.
Uso en producción.	Bajo.	Alto.

EJEMPLO: ANÁLISIS INTEGRADO EN R Y PYTHON

- Una empresa del sector retail desea optimizar sus estrategias de marketing y fidelización de clientes.
- Para ello, el área de analítica solicita evaluar si existen diferencias significativas en el gasto mensual promedio de sus clientes según el género, con el fin de diseñar campañas segmentadas y asignar mejor su presupuesto promocional.
- La base de datos analizada corresponde a una muestra de clientes frecuentes e incluye dos variables principales:
 - **Género del cliente (hombre, mujer).**
 - **Gasto mensual promedio (en soles).**



EJEMPLO: ANÁLISIS INTEGRADO EN R Y PYTHON

- El objetivo del análisis es determinar si el gasto promedio difiere significativamente entre ambos grupos y, a partir de ello, generar información útil para la toma de decisiones comerciales.
- Para este fin, **se emplean Python y R de forma integrada**, aprovechando las fortalezas de cada herramienta dentro de un mismo flujo analítico.



+ PROGRAMACIÓN EN PYTHON

```
import pandas as pd
from scipy import stats

# Creación del conjunto de datos
clientes = pd.DataFrame({
    "genero": ["Hombre", "Hombre", "Mujer", "Mujer", "Hombre", "Mujer"],
    "gasto": [450, 520, 600, 580, 480, 610]
})

# Análisis descriptivo: gasto promedio por género
promedios = clientes.groupby("genero")["gasto"].mean()
print(promedios)

# Prueba estadística t de Student
hombres = clientes[clientes["genero"] == "Hombre"]["gasto"]
mujeres = clientes[clientes["genero"] == "Mujer"]["gasto"]

resultado_t = stats.ttest_ind(hombres, mujeres)
print(resultado_t)
```

+ PROGRAMACIÓN EN R

```
clientes <- data.frame(  
  genero = c("Hombre", "Hombre", "Mujer", "Mujer", "Hombre", "Mujer"),  
  gasto = c(450, 520, 600, 580, 480, 610)  
)  
  
# Análisis descriptivo  
aggregate(gasto ~ genero, data = clientes, mean)  
  
# Prueba t de Student  
t.test(gasto ~ genero, data = clientes)
```

+ SALIDA DEL ANÁLISIS DESCRIPTIVO EN PYTHON

```
genero
Hombre    483.33
Mujer     596.67
Name: gasto, dtype: float64
```

+ SALIDA DE LA PRUEBA T DE STUDENT

```
Ttest_indResult(statistic=-3.90, pvalue=0.018, df=4)
```

INTERPRETACIÓN DEL RESULTADO

R

- El valor más importante del análisis inferencial es el p-value = 0.018.
- Como este valor es menor que 0.05, se concluye que la diferencia en el gasto promedio entre hombres y mujeres es estadísticamente significativa; es decir, no se debe al azar.



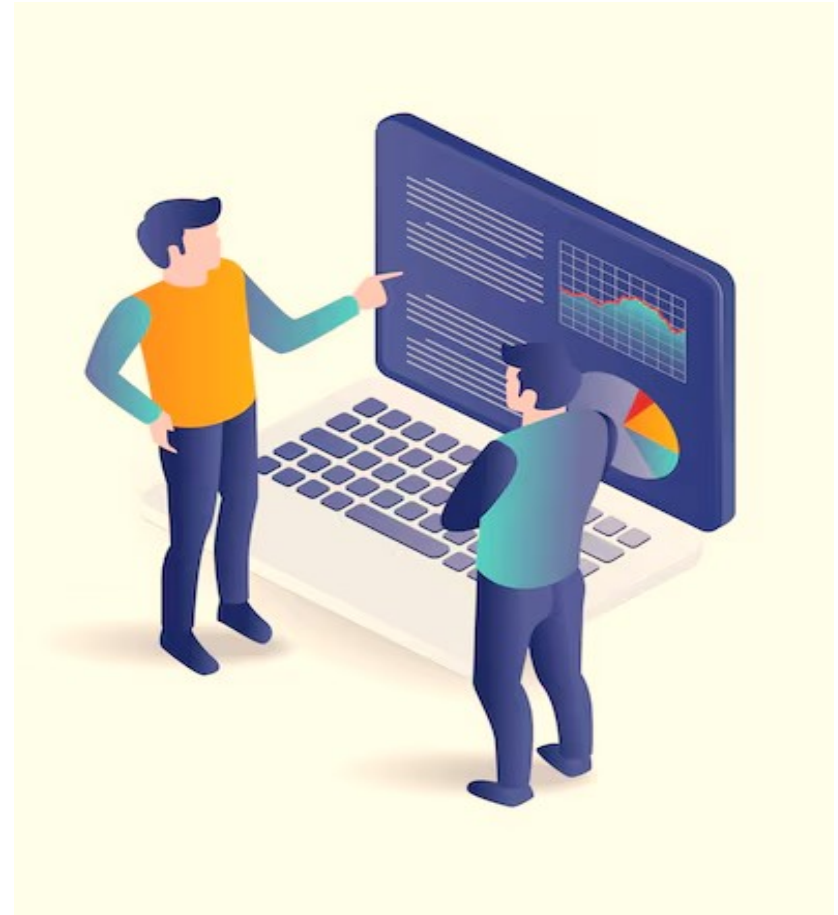
PYTHON

- Muestra que el gasto promedio mensual de las mujeres (S/596.67) es mayor que el de los hombres (S/483.33).
- Esta diferencia, a nivel descriptivo, ya sugiere un comportamiento distinto entre ambos grupos.

INTERPRETACIÓN EN EL CONTEXTO DEL NEGOCIO

Desde el punto de vista del negocio, este resultado indica lo siguiente:

- El género influye en el nivel de gasto mensual de los clientes.
- Las mujeres representan un segmento de mayor valor económico promedio.
- La empresa puede justificar:
 - Campañas diferenciadas por género.
 - Programas de fidelización enfocados en el segmento femenino.
 - Ajustes en promociones y comunicación comercial.



+

INTEGRACIÓN: COMBINAR
FLUJOS DE TRABAJO

INTEGRACIÓN DE FLUJOS DE TRABAJO

- Consiste en utilizar más de una herramienta dentro de un mismo proceso de análisis de datos, de manera ordenada y coherente, para aprovechar las fortalezas específicas de cada lenguaje.
- En análisis estadístico y minería de datos, la integración más común es la siguiente:

R

- Como herramienta complementaria para análisis estadístico especializado, validación de resultados o visualización avanzada.

PYTHON

- Como lenguaje principal del flujo (gestión, limpieza, automatización, conexión con sistemas).

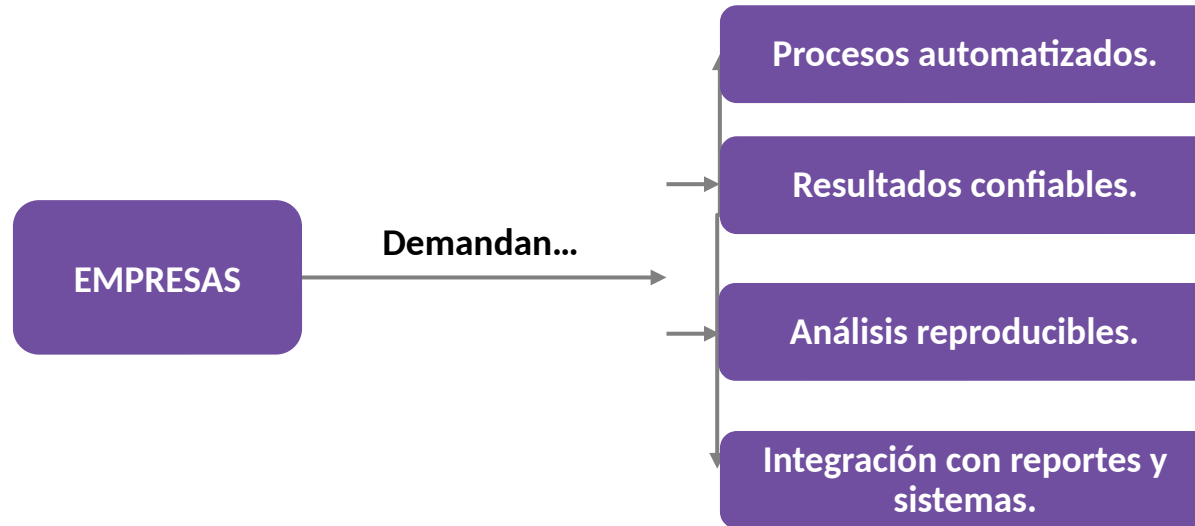
INTEGRACIÓN DE FLUJOS DE TRABAJO

- La integración no implica duplicar trabajo, sino **coordinar tareas**, para permitir que los resultados generados en un lenguaje sean utilizados por el otro.
- La integración de Python y R no es redundante, es estratégica. **Python construye el proceso; R fortalece el análisis.**



INTEGRACIÓN EN EL ENTORNO LABORAL

En el entorno laboral real, los proyectos de datos no se resuelven con una sola herramienta.



INTEGRACIÓN EN EL ENTORNO LABORAL

- Python es ideal para construir el pipeline completo, mientras que R aporta rigor estadístico.
- La integración permite:
 - **Reducir tiempos de análisis.**
 - **Validar resultados.**
 - **Mejorar la calidad de las decisiones.**
 - **Trabajar en equipos multidisciplinares.**



FLUJO INTEGRADO

Un flujo integrado típico sigue la siguiente lógica:



En conclusión, Python lidera el proceso; R apoya el análisis estadístico.

EJEMPLO: PROCESO DE DESARROLLO INTEGRADO

- Una entidad financiera desea evaluar si el ingreso mensual promedio de sus clientes difiere según el tipo de producto financiero adquirido (cuenta básica o cuenta premium).
- Esta información permitirá ajustar estrategias comerciales y segmentar mejor a sus clientes.
- La base de datos contiene:
 - **Tipo de cuenta.**
 - **Ingreso mensual del cliente (\$/).**
- El análisis se desarrolla en un flujo integrado Python-R para asegurar eficiencia y validación estadística.



+ PROGRAMACIÓN EN PYTHON

```
import pandas as pd
from scipy import stats

# Creación del conjunto de datos
clientes = pd.DataFrame({
    "cuenta": ["Basica", "Basica", "Premium", "Premium", "Basica", "Premium"],
    "ingreso": [2500, 2700, 4200, 4000, 2600, 4500]
})

# Análisis descriptivo inicial
promedios = clientes.groupby("cuenta")["ingreso"].mean()
print(promedios)

# Prueba estadística
basica = clientes[clientes["cuenta"] == "Basica"]["ingreso"]
premium = clientes[clientes["cuenta"] == "Premium"]["ingreso"]

resultado_t = stats.ttest_ind(basica, premium)
print(resultado_t)
```

+ PROGRAMACIÓN EN R

```
clientes <- data.frame(  
  cuenta = c("Basica", "Basica", "Premium", "Premium", "Basica", "Premium"),  
  ingreso = c(2500, 2700, 4200, 4000, 2600, 4500)  
)  
  
# Análisis descriptivo  
aggregate(ingreso ~ cuenta, data = clientes, mean)  
  
# Prueba t de Student  
t.test(ingreso ~ cuenta, data = clientes)
```

+ SALIDA DEL ANÁLISIS DESCRIPTIVO EN PYTHON

```
cuenta  
Basica      2600.00  
Premium     4233.33  
Name: ingreso, dtype: float64
```

+ SALIDA DE LA PRUEBA T DE STUDENT

```
Ttest_indResult(statistic=-6.12, pvalue=0.004, df=4)
```

RESULTADOS

- Python muestra que los clientes con cuenta premium tienen un ingreso promedio significativamente mayor.
- R muestra que el $p\text{-value} = 0.004$, menor a 0.05, indica que esta diferencia no es aleatoria.

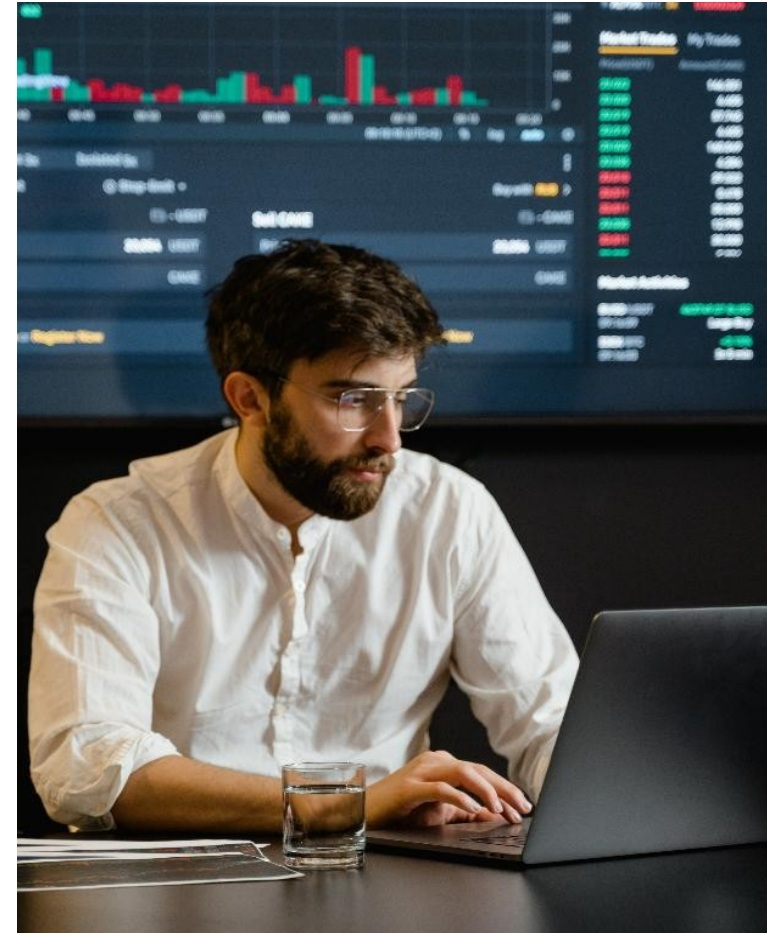
Podemos concluir que existe una diferencia estadísticamente significativa en el ingreso mensual según el tipo de cuenta.



INTERPRETACIÓN EN EL CONTEXTO DEL NEGOCIO

Desde la perspectiva de la entidad financiera:

- El tipo de cuenta está asociado al nivel de ingresos del cliente.
- Los clientes premium representan un segmento de mayor capacidad económica.
- Se pueden diseñar:
 - **Productos exclusivos.**
 - **Ofertas de inversión diferenciadas.**
 - **Estrategias de fidelización específicas.**
- La integración R-Python asegura que la decisión se base en resultados confiables y validados.

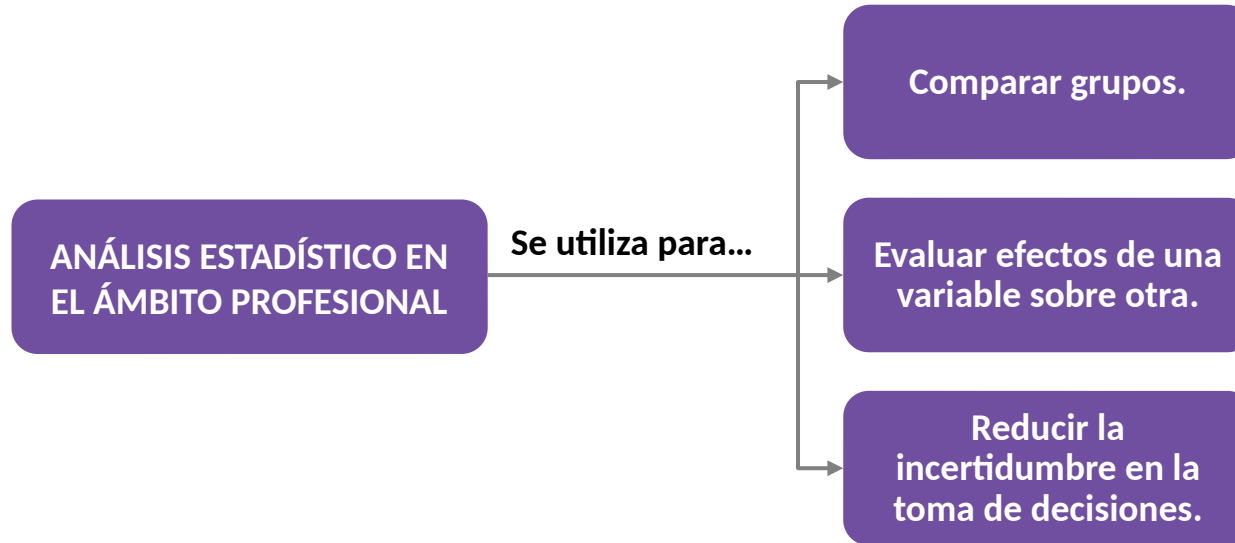


+

ANÁLISIS ESTADÍSTICO:
**PRUEBAS EN
AMBAS HERRAMIENTAS**

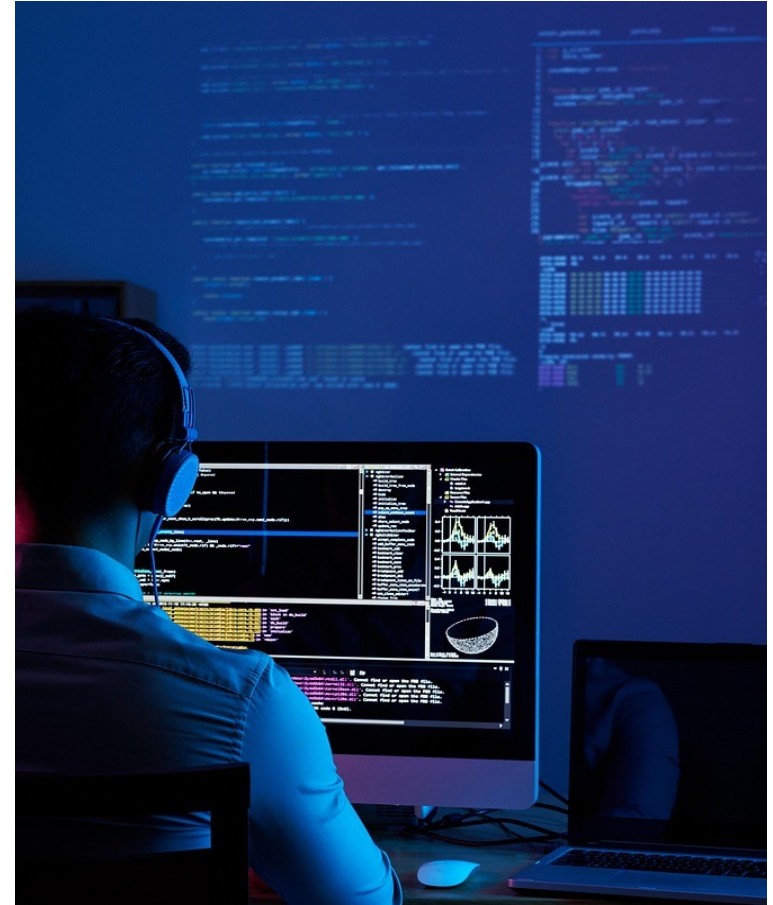
ANÁLISIS ESTADÍSTICO

- Es el conjunto de métodos que permite interpretar datos y extraer conclusiones confiables, apoyando la toma de decisiones en contextos académicos y empresariales.
- A través del análisis estadístico, los datos dejan de ser simples números y se convierten en información útil.



PYTHON Y R EN ANÁLISIS ESTADÍSTICOS

- En proyectos reales de análisis de datos, Python se utiliza como el lenguaje central del flujo de trabajo al integrar lo siguiente:
 - **Gestión de datos.**
 - **Aplicación de pruebas estadísticas.**
 - **Automatización de resultados.**
 - **Integración con reportes y sistemas.**
- El análisis estadístico en **Python** permite validar la hipótesis de negocio antes de implementar acciones costosas o riesgosas.
- **R** complementa este proceso al ofrecer mayor detalle y claridad estadística, especialmente en la interpretación inferencial.



PRUEBAS ESTADÍSTICAS

Ayudan a responder preguntas específicas sobre los datos. Algunas de las más utilizadas en entornos profesionales son:

Prueba t de Student	ANOVA	Correlación	Regresión
Compara el promedio de dos grupos.	Compara el promedio de tres o más grupos.	Mide la relación entre dos variables.	Analiza el efecto de una variable sobre otra.

Estas pruebas están disponibles tanto en Python como en R, aunque con enfoques ligeramente distintos.

DIFERENCIAS ENTRE PYTHON Y R EN ANÁLISIS ESTADÍSTICO

ASPECTO	PYTHON	R
Rol principal.	Desarrollo y automatización.	Profundidad estadística.
Uso en empresas.	Alto.	Medio.
Curva de aprendizaje.	Progresiva.	Más técnica.
Presentación de resultados.	Clara y funcional.	Detallada y analítica.
Integración con sistemas.	Excelente.	Limitada.

+

MINERÍA DE DATOS:
**ALGORITMOS EN R
Y PYTHON**

MINERÍA DE DATOS

- Es el proceso mediante el cual se analizan grandes volúmenes de datos con el objetivo de descubrir patrones, relaciones y comportamientos ocultos que no son evidentes a simple vista.
- A diferencia del análisis estadístico tradicional, **la minería de datos se centra en predicción, segmentación y clasificación.**
- En el contexto empresarial, la minería de datos permite lo siguiente:
 - **Anticiparse a situaciones futuras.**
 - **Comprender el comportamiento de clientes.**
 - **Optimizar procesos de decisión.**



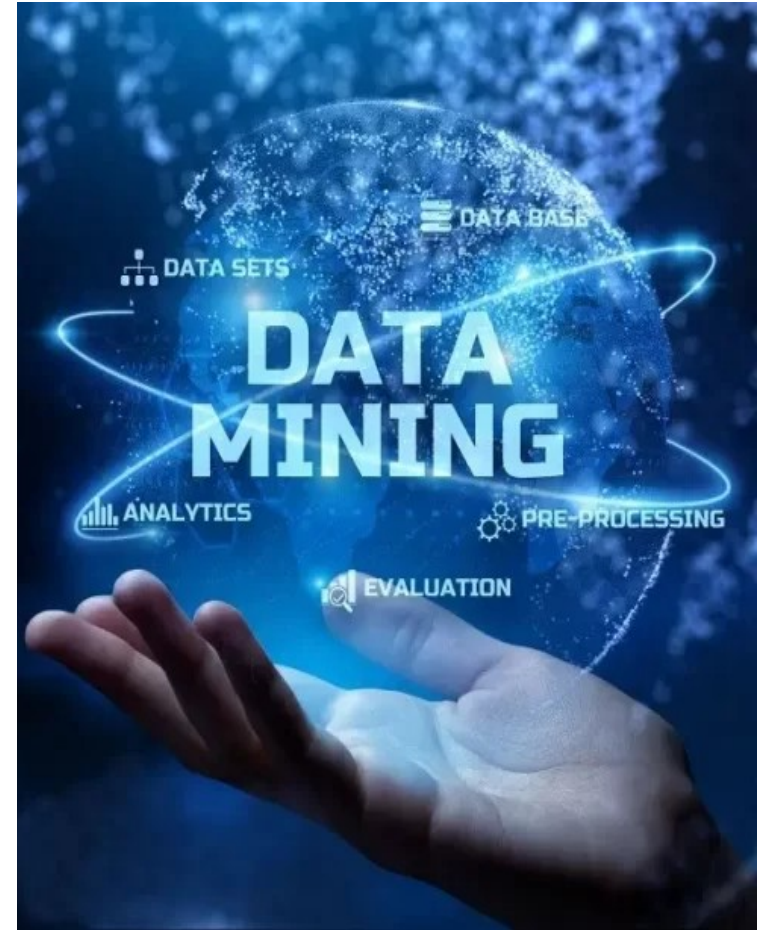
¿EN QUÉ SECTORES SE UTILIZA LA MINERÍA DE DATOS?

La minería de datos es ampliamente utilizada en sectores como los siguientes:



IMPORTANCIA DE LA MINERÍA DE DATOS

- + Tomar decisiones basadas en datos históricos.
- + Detectar oportunidades y riesgos.
- + Automatizar procesos de análisis.
- + Mejorar la competitividad de las organizaciones



ALGORITMOS DE MINERÍA DE DATOS

Se agrupan según el objetivo del análisis:

+ 01.

ALGORITMOS DE CLASIFICACIÓN

Se utilizan cuando se desea asignar una categoría o clase a un conjunto de datos. Por ejemplo:

- Clasificar clientes como “riesgosos” o “no riesgosos”.
- Determinar si un cliente abandonará o no un servicio.



ALGORITMOS DE MINERÍA DE DATOS

Se agrupan según el objetivo del análisis:

+ 02.

ALGORITMOS DE CLUSTERING (SEGMENTACIÓN)

Se utilizan para agrupar datos similares sin categorías predefinidas.
Por ejemplo:

- Segmentación de clientes según comportamiento de compra.
- Agrupación de productos por nivel de ventas.



ALGORITMOS DE MINERÍA DE DATOS

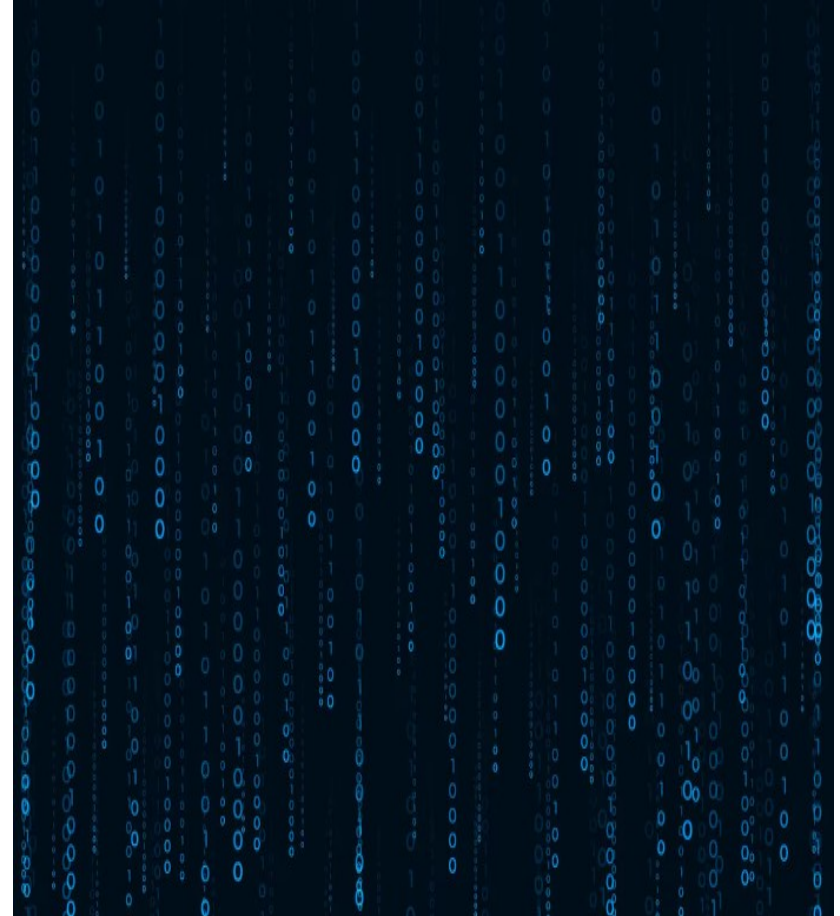
Se agrupan según el objetivo del análisis:

+ 03.

ALGORITMOS DE REGRESIÓN

Permiten predecir valores numéricos a partir de variables explicativas. Por ejemplo:

- Predicción de ventas.
- Estimación de precios o costos.



DIFERENCIAS Y CARACTERÍSTICAS DE LA MINERÍA DE DATOS EN R Y PYTHON

ASPECTO	R	PYTHON
Enfoque principal.	Análisis estadístico	Desarrollo y producción.
Uso en minería de datos.	Académico y analítico.	Empresarial y escalable.
Librerías principales.	Caret, RandomForest.	Scikit-learn.
Automatización.	Limitada.	Alta.
Integración con sistemas.	Media.	Excelente.

PROCESO DE MINERÍA DE DATOS

Independientemente de la herramienta utilizada, el proceso de minería de datos sigue una estructura común:

- 01  Comprensión del problema.
- 02  Preparación de los datos.
- 03  Selección del algoritmo
- 04  Entrenamiento del modelo.
- 05  Evaluación de resultados.
- 06  Interpretación y uso del modelo.

Este proceso puede ejecutarse tanto en R como en Python, con ligeras diferencias en implementación.

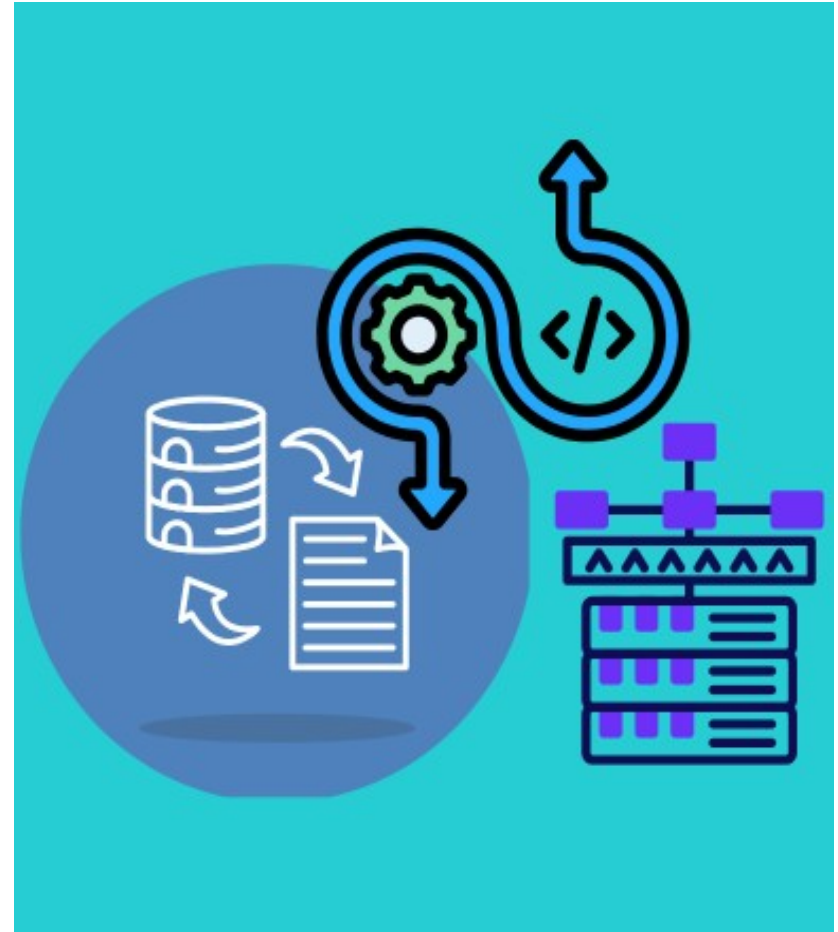


+

OPTIMIZACIÓN DE FLUJOS DE
**TRABAJO: EFICIENCIA EN
PIPELINES DE DATOS**

OPTIMIZACIÓN DE FLUJOS DE TRABAJO

- En análisis de datos, **consiste en diseñar y automatizar una secuencia ordenada de procesos que permiten transformar datos crudos en información útil de manera eficiente**, reproducible y escalable. A esta secuencia se le denomina pipeline de datos.
- Un pipeline de datos reduce la intervención manual y minimiza errores al integrar etapas como las siguientes:
 - **Ingesta de datos.**
 - **Limpieza de datos.**
 - **Transformación y preparación.**
 - **Análisis estadístico y modelamiento.**
 - **Automatización y ejecución.**
 - **Visualización y reporte.**



OPTIMIZACIÓN DE PIPELINES

Es clave en organizaciones modernas por lo siguiente:

- **Reduce el tiempo de análisis** y procesamiento de información.
- **Garantiza la reproducibilidad de los resultados**, esencial en auditorías y reportes gerenciales.
- **Permite trabajar con grandes volúmenes de datos** de forma ordenada.
- **Facilita la colaboración** entre analistas de datos y tomadores de decisiones.

Un pipeline bien diseñado permite obtener reportes periódicos confiables y tomar decisiones oportunas basadas en datos actualizados.



ETAPAS DEL DESARROLLO DE UN PIPELINE DE DATOS

+ 01. INGESTA DE DATOS

- Consiste en obtener los datos desde diversas fuentes:
 - Archivos Excel o CSV.
 - Bases de datos empresariales.
 - Sistemas internos de ventas o contabilidad.
- **En R:** uso de paquetes como readr, readxl, DBI.
- **En Python:** uso de pandas, sqlalchemy.

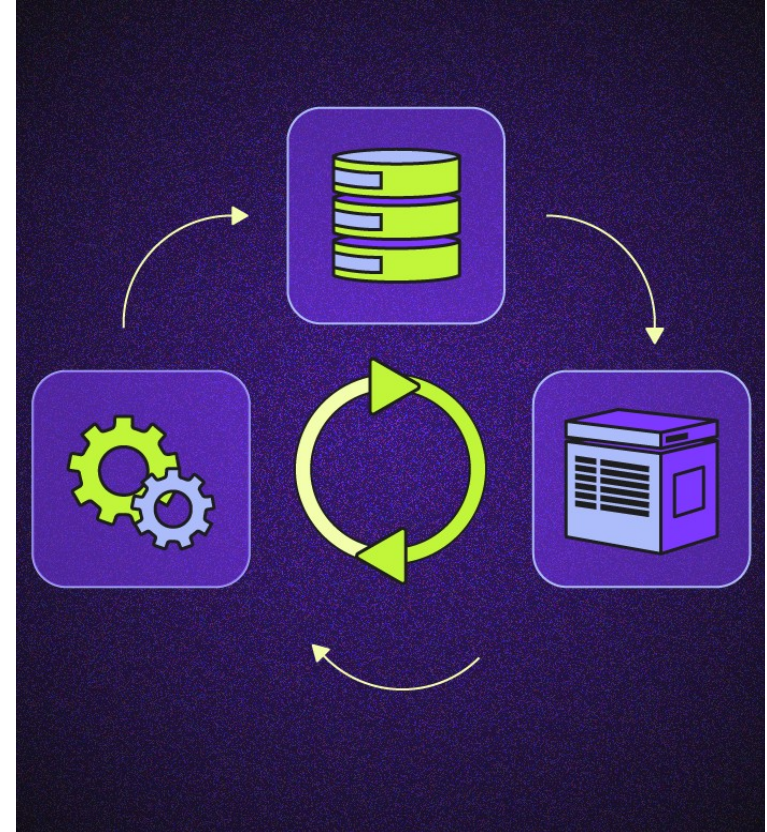


ETAPAS DEL DESARROLLO DE UN PIPELINE DE DATOS

+ 02.

LIMPIEZA DE DATOS

- Se corrigen errores y se preparan los datos para el análisis:
 - Eliminación de valores nulos o duplicados.
 - Corrección de formatos (fechas, monedas).
 - Normalización de variables.
- **En R:** funciones de dplyr y tidyr.
- **En Python:** métodos de pandas.
- Esta fase es crítica, ya que datos mal preparados generan conclusiones incorrectas.

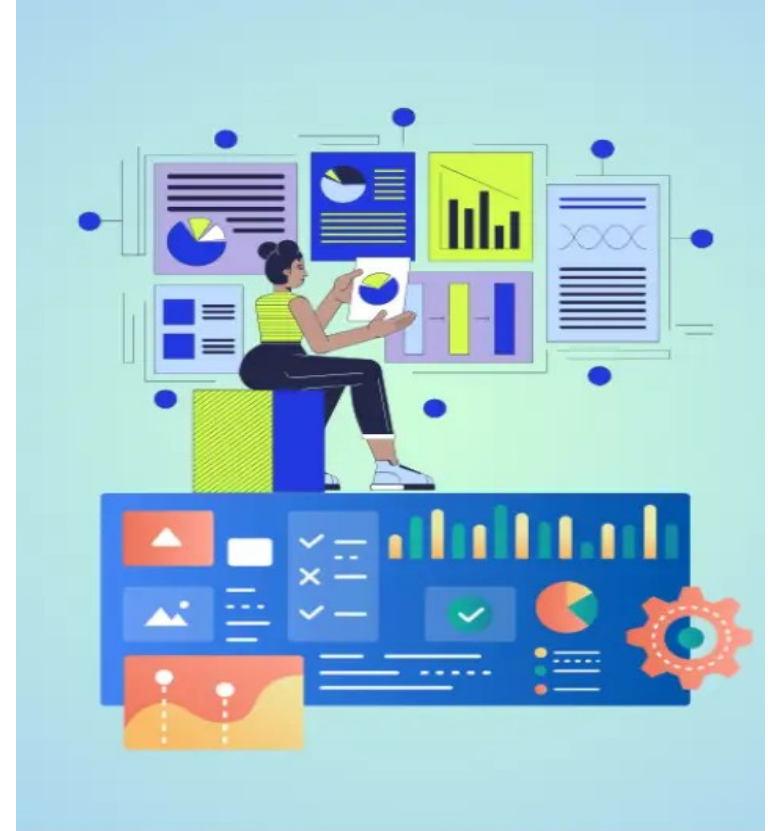


ETAPAS DEL DESARROLLO DE UN PIPELINE DE DATOS

+ 03.

TRANSFORMACIÓN Y PREPARACIÓN

- Los datos se reorganizan para facilitar el análisis:
 - Creación de nuevas variables.
 - Agrupación y agregación de información.
 - Filtrado según criterios del negocio.
- **En R:** uso de pipes (`%>%`) con dplyr.
- **En Python:** encadenamiento de métodos con pandas.
- Aquí se define la estructura final del dataset analítico.



ETAPAS DEL DESARROLLO DE UN PIPELINE DE DATOS

+ 04.

ANÁLISIS ESTADÍSTICO Y MODELAMIENTO

- Se aplican técnicas estadísticas o algoritmos de minería de datos:
 - Estadística descriptiva.
 - Pruebas de hipótesis.
 - Modelos predictivos.
- **En R:** paquetes estadísticos especializados.
- **En Python:** scikit-learn, statsmodels.
- Esta etapa convierte los datos en conocimiento útil.



ETAPAS DEL DESARROLLO DE UN PIPELINE DE DATOS

+ 05.

AUTOMATIZACIÓN Y EJECUCIÓN

- El pipeline se programa para ejecutarse de forma automática:
 - Scripts programados.
 - Notebooks reproducibles.
 - Integración con sistemas de reporte.
- **En R:** .R, R Markdown.
- **En Python:** scripts .py, Jupyter Notebooks.

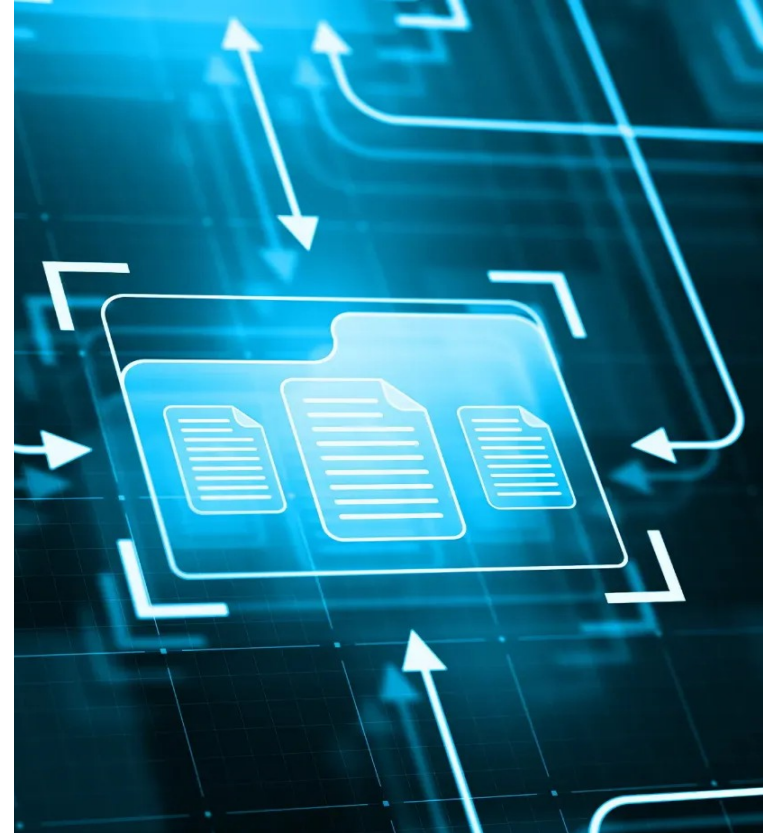


ETAPAS DEL DESARROLLO DE UN PIPELINE DE DATOS

+ 06.

VISUALIZACIÓN Y REPORTE

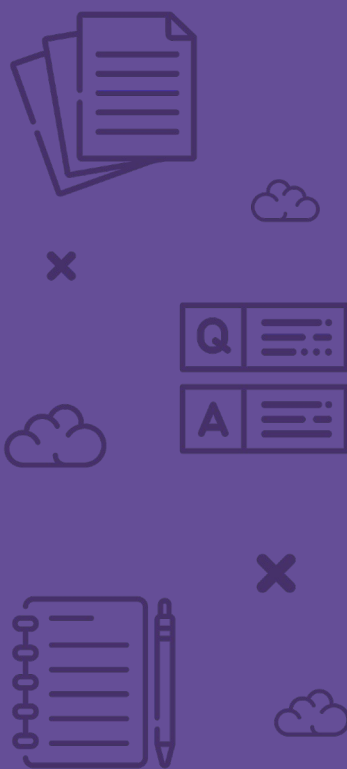
- Los resultados se presentan de forma clara para la toma de decisiones:
 - Gráficos.
 - Tablas dinámicas.
 - Informes ejecutivos.
- **En R:** ggplot2, flexdashboard.
- **En Python:** matplotlib, seaborn, dash.



DIFERENCIAS Y CARACTERÍSTICAS DE R Y PYTHON EN PIPELINES DE DATOS

ASPECTO	R	PYTHON
Enfoque principal.	Estadístico y analítico.	General y automatización.
Manejo de datos.	Muy eficiente con dplyr.	Flexible con pandas.
Automatización.	Ideal para análisis repetitivos.	Ideal para procesos complejos.
Reportes.	R Markdown.	Notebooks y dashboards.
Integración.	Fuerte en análisis.	Fuerte en sistemas empresariales.

Ambos lenguajes son complementarios y pueden integrarse dentro de un mismo pipeline.

A collection of white line-art icons on a purple background. At the top left is a stack of three papers. Below it is a small 'x' icon. To the right of the 'x' is a cloud icon. Below the cloud is a Q&A box with 'Q' and 'A' sections, each containing three horizontal lines. To the left of the Q&A box is another cloud icon. At the bottom left is a spiral notebook with a pencil. To its right is a small 'x' icon. At the bottom right is a cloud icon.

+ **CONCLUSIONES**

+ CONCLUSIONES

- + El análisis integrado en R y Python representa una práctica clave en la analítica de datos moderna al permitir combinar la solidez estadística de R con la versatilidad y capacidad de automatización de Python.
- + Esta integración favorece el desarrollo de análisis más completos, reproducibles y escalables, alineados con las exigencias actuales de las organizaciones orientadas a la toma de decisiones basada en datos.
- + En los entornos profesionales, el dominio de flujos de trabajo integrados con R y Python fortalece la eficiencia operativa, mejora la calidad del análisis y optimiza los procesos de generación de información estratégica.
- + La capacidad de diseñar pipelines de datos bien estructurados se ha convertido en un factor diferenciador, ya que permite responder con rapidez a cambios del mercado, reducir errores analíticos y transformar grandes volúmenes de datos en conocimiento útil para la gestión y la planificación organizacional.





x



x



+ BIBLIOGRAFÍA **MÁS REFERENCIAS**

- + Betancourt, D. (2025). *Análisis de datos con Python: Data Science Nivel 1* (Edición en español). Editorial Independiente.
- + Bruce, P., Bruce, A., & Gedeck, P. (2022). *Estadística práctica para ciencia de datos con R y Python* (2ª ed.). Marcombo, S. A.
- + Fernández-Avilés, G., & Montero, J. M. (Eds.). (2024). *Fundamentos de ciencia de datos con R*. Editorial datos.gob.es.
- + Padilla Beltrán, J. E., & Contreras Bravo, L. E. (2024). *Ciencia de datos con Python: Transformación y selección de variables*. Ediciones de la U.



ISIL+